SIWES TRACKER · THE COMPLETE GUIDE

Every line, one lesson

The whole app in a single document: how the machine runs, what every line of the 12 PHP files does, why each security measure exists, and the answers to the questions a viva would ask.

Scope: all core PHP + `database.sql` · Python `backend/` folder is **not part of this app** and never mentioned again · work through it in sittings, one part at a time; each part ends with a from-memory checkpoint

Contents

1. The machine: what "running a PHP file" means
2. config.php — the foundation, line by line
3. helpers.php — the toolbox, function by function
4. The partials — icons, layout-top, layout-bottom
5. database.sql — the schema and every constraint
6. login.php, register.php, logout.php — authentication
7. log_form.php, logs.php, log_delete.php, profile.php — CRUD
8. dashboard.php — stats, arrays, the pure-CSS chart
9. seed.php & router.php — the support files
10. The security map — the five defences, and the attacks they stop
11. Viva drill — 14 questions, with model answers
12. Final exam — 12 mixed questions (no scrolling back)

1 The machine: what "running a PHP file" means

Everything else in this document only makes sense on top of one idea, so we start there: **in this app, a file is a program the web server runs once per browser request.** You never "launch" the app. Each URL request wakes a file up, it computes, prints HTML, and dies.

1. Your browser sends an HTTP request: `GET /siwes%20tracker/dashboard.php`.
2. **Apache** (XAMPP's web server) maps that URL to the folder on disk and sees the `.php` extension. It does *not* send the file to the browser — it hands it to the **PHP interpreter**.

3. PHP executes the file **top to bottom**. Text outside `<?php … ?>` tags is copied verbatim into the output buffer; text inside them *runs*. (PHP tags — Manual)

4. When the file finishes, Apache packages whatever was printed as the HTTP response. **The browser never receives PHP source code** — only HTML.

Because the script dies after every request, there is no "while the app is open" — which raises the central question of the whole course: *where does state live between requests?* Exactly two answers:

Store	Lives in	Used here for
MySQL	XAMPP's MySQL server, permanent	Students, passwords (hashed), log entries
`$_SESSION`	A file on the server, keyed to a browser **cookie**	"who is logged in", flash messages, CSRF token

The session trick is the one to internalise now (full detail in part 2). When `session_start()` runs, PHP reads an incoming cookie (`PHPSESSID=abc123…`) and loads the server-side file `sess_abc123…` into `$_SESSION`. At the end of the request, whatever you put into `$_SESSION` is saved back. So `$_SESSION['student_id'] = 5` in login.php and reading it in dashboard.php isn't memory — it's a *message passed through the disk between two separate runs*.

> **Two consequences that explain most of the code you'll see:** (1) every page re-does its setup from zero — that's why every file starts with `require_once __DIR__ . '/helpers.php';` ; (2) sending the user somewhere means telling *their browser* to start a new request — `header('Location: …')` then `exit` — which is why `redirect()` can only be called *before any HTML is printed*: headers ship out at the start of the response, before the body.

Hands-on checkpoint: in XAMPP with the app open, press F12 → Network tab, log out via the sidebar. You should see `logout.php` return status **302** with an empty response body, then an automatic second request `login.php` (200). That 302 *is* `redirect()` in the wild.

Who executes the PHP in dashboard.php?

- The PHP interpreter on the server
- The JavaScript engine in the browser
- The MySQL query processor on disk

Why can redirect() only run before any HTML prints?

- Headers ship before the body does
- PHP disables functions after output
- Apache rejects late redirects always

After a request finishes, where does a variable's value survive?

- Nowhere — only session and database
- Anywhere — PHP keeps globals warm
- Only in cookies the browser can read

---

2 config.php — the foundation, line by line

Every page in the app eventually loads `config.php` (via `helpers.php`). Its 31 lines do four jobs: declare strict typing, publish settings as constants, start the session safely, and fix the clock. Here it is whole, then dissected.

```
<?php
declare(strict_types=1);

define('DB_HOST', '127.0.0.1');
define('DB_NAME', 'siwes_tracker');
define('DB_USER', 'root');
define('DB_PASS', '');          // XAMPP default: empty password
define('DB_PORT', 3306);

define('APP_NAME', 'SIWES Tracker');
define('APP_URL', '/siwes%20tracker');

if (session_status() === PHP_SESSION_NONE) {
    session_set_cookie_params([
        'httponly' => true,
        'samesite' => 'Lax',
    ]);
    session_start();
}
```

```
date_default_timezone_set('Africa/Lagos');
```

`declare(strict_types=1);` (line 7)

Opt this *file* into strict type checking: pass an int where a `string` parameter is declared and PHP throws instead of quietly converting. It's **per-file** — which is why login.php (which has no declare) behaves slightly differently from helpers.php (which does). The app's style guide is in one line, and it matches PHP The Right Way's recommendation. (Type declarations — Manual)

The seven `define()` lines (10–18)

`define('DB_HOST', '127.0.0.1')` creates a *constant*: an uppercase global name, `DB_HOST`, whose value **no code can overwrite later**. Constants vs variables — `DB_HOST` (no `$`) can't be reassigned mid-request; `$x` can. Why `127.0.0.1` and not `localhost`: `localhost` makes MySQL try a local named-pipe/socket first; the numeric IP forces a normal TCP connection — deterministic on XAMPP on Windows.

`DB_PASS` is empty because that's XAMPP's stock `root`. That's fine *here* — dev machine, no external access — but is exactly the kind of line a viva will point at: on a real server, never the root user, never an empty password, never the credentials in the repo.

Dead code find: `APP_URL` (line 18) is defined but *used nowhere* — grep the app and you'll only ever see `APP_NAME`. It's a leftover; every link in the app is relative (`href="logs.php"`), which is why moving the folder doesn't break anything. Knowing your own files well enough to call out their vestigial bits is viva gold.

The session block (21–27)

Read it inside-out:

- `session_status() === PHP_SESSION_NONE` — "has a session *not* started yet?" The guard exists because config.php might run after something else already called `session_start()`; starting twice would throw a warning.
- `session_set_cookie_params([...])` — configures the **cookie** that identifies this browser's session. Two flags, both security: `httponly` = JavaScript in the browser cannot read the cookie, so an XSS bug can't steal the session id via script; `samesite Lax` = the cookie is withheld on cross-site *POSTs*, so another website can't silently submit a form to your app using this visitor's session. Both are defence-in-depth behind part 10's CSRF token. (Manual)
- `session_start()` — **must** come after setting the cookie params (they're read at start time, not retroactively), and it's the line that populates `$_SESSION` for the whole request. This is

why config.php must load before any page reads `$_SESSION` — and why `require_once` is line 1 of the chain.

`date_default_timezone_set('Africa/Lagos')` (line 30)

Every `date()` and `strtotime()` call in the app — today's default in the log form, the friendly day labels, the week maths — reads this. Without it PHP defaults to UTC and would show "tonight" as "tomorrow" after 11pm. One line prevents an entire class of timezone bugs.

Why are the DB settings constants rather than variables?

- Constants cannot get overwritten later
- Constants load faster than variables
- Constants hide values from other files

Why must session_set_cookie_params() precede session_start()?

- The cookie is built at start, once
- Sessions lock after any first read
- PHP forbids params after two hundred

What do 'httponly' and 'samesite' defend against?

- Cookie theft and cross-site form posts
- Slow queries and dead server connections
- Duplicate accounts and forgotten passwords

---

3 helpers.php — the toolbox, function by function

121 lines, 11 functions, loaded by every page. This file *is* the app's framework — understanding it is understanding the whole system. It begins:

```php
<?php
declare(strict_types=1);

require_once __DIR__ . '/config.php';
require_once __DIR__ . '/partials/icons.php';
```

`__DIR__` = the absolute path of *this file's* directory — so the require works no matter where the server was started from. `require_once` = load and run it now; if it's already loaded in this request, skip silently. The `_once` matters because `icons.php` also needs helpers, and pages require helpers directly *and* through the layout — without the guard, PHP would redeclare `db()` and fatal.

`db()` — the only door to MySQL (lines 12–24)

```php
function db(): PDO
{
    static $pdo = null;
    if ($pdo === null) {
        $dsn = 'mysql:host=' . DB_HOST . ';port=' . DB_PORT . ';dbname=' . DB_NAME . ';charset=utf8mb4';
        $pdo = new PDO($dsn, DB_USER, DB_PASS, [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES   => false,
        ]);
    }
    return $pdo;
}
```

Every database line in the app starts with `db()`. Four ideas packed into one function:

- **The `static` trick.** A `static` variable inside a function survives between *calls* (within one request). First call: `$pdo === null`, build the connection. Every later call: return the same object. This is a *lazy singleton* — one connection per request, created only if some code actually needs the DB. logout.php never queries the DB and never pays for a connection.
- **The DSN.** One string describing *where* to connect: driver, host, port, database, charset. `charset=utf8mb4` guarantees text arrives as true UTF-8 (and, historically, disables a charset-confusion injection vector).
- **The three options.** `ERRMODE_EXCEPTION`: a bad query throws a catchable exception instead of failing silently — the seed.php try/catch works because of this line. `FETCH_ASSOC`: rows come back as arrays keyed by column *name* — why you write `$row['work_done']` everywhere. `EMULATE_PREPARES => false`: don't fake prepared statements by string-building

in PHP — send them to MySQL's real prepare/execute protocol, which is what makes `?` placeholders genuinely injection-proof. (PDO — Manual)
- **Types.** The signature `function db(): PDO` promises a PDO object back; with strict_types in this file, a violation is fatal, not sloppy.

The whole app's DB usage is then just the same three-step dance (prepare → execute → fetch):

```
$stmt = db()->prepare('SELECT * FROM students WHERE email = ?');
$stmt->execute([$email]);
$student = $stmt->fetch();        // one row, or false if none
```

The `?` marks a **placeholder**: SQL structure is sent to MySQL first, the user's data separately afterwards. The data can *never* be parsed as SQL — that's why injection is impossible here, even though `$email` comes straight from a form. `fetch()` returns the next row (array) or `false`; `fetchAll()` returns all remaining rows as an array-of-arrays; `fetchColumn()` returns one scalar (you'll meet it in `siwes_week_number`).

`e()` — the output-escaping one-liner (27–30)

```
function e(?string $value): string
{
    return htmlspecialchars((string)$value, ENT_QUOTES, 'UTF-8');
}
```

`?string` = accepts a string *or null*; `(string)` coerces (null → ""), and htmlspecialchars replaces `< > & " '` with entities like `<` (`ENT_QUOTES` = also single quotes). The effect: if a student types `<script>alert(1)</script>` into "work done", it is stored as text, and on output rendered as *visible text* instead of executing. This is the second great defence (XSS), and this six-letter function is why 200+ HTML lines across the app stay safe. Rule the codebase follows everywhere: **data comes out of the DB dirty; it becomes HTML only through `e()`.**

Auth helpers (33–66)

```
function redirect(string $path): void
{
    header('Location: ' . $path);
    exit;
}

function is_logged_in(): bool
{
    return isset($_SESSION['student_id']);
```

```
}

function require_login(): void
{
    if (!is_logged_in()) {
        $_SESSION['flash_error'] = 'Please log in to continue.';
        redirect('login.php');
    }
}
```

- `redirect()` : the part-1 lifecycle rule in code — send the 302, then `exit` so no stray code runs after we've told the browser to leave. Every `: void` return means "returns nothing".
- `is_logged_in()` : the entire authentication model is *one key in one array*. "Logged in" = a prior request wrote `student_id` into this session.
- `require_login()` : the gate, called as line 3 of every protected page. Note it writes `$_SESSION['flash_error']` directly rather than calling `flash('error', …)` — same key, just skipping the helper (a small inconsistency you're allowed to notice).

```
function current_student(): ?array
{
    static $student = null;
    static $loaded  = false;
    if (!$loaded && is_logged_in()) {
        $stmt = db()->prepare('SELECT * FROM students WHERE id = ?');
        $stmt->execute([$_SESSION['student_id']]);
        $student = $stmt->fetch() ?: null;
        $loaded  = true;
    }
    return $student;
}
```

The static-cache pattern again, twice over: `$loaded` ensures "checked the DB" is remembered even if the result was `no row` (`?:` — the elvis — turns `false` from fetch into `null`). A page like dashboard.php calls `current_student()` and gets a DB hit *once*, even if ten places ask. The `?array` return type: array or null, because in theory the session can name a student row that no longer exists (deleted account) — and `layout-top.php` is written to survive `null` (part 4).

`flash()` — the one-shot message system (69–78)

```
function flash(string $key, ?string $message = null): ?string
{
    if ($message !== null) {
        $_SESSION['flash_' . $key] = $message;
```

```
        return null;
    }
    $value = $_SESSION['flash_' . $key] ?? null;
    unset($_SESSION['flash_' . $key]);
    return $value;
}
```

One function, two modes — the signature gives it away: `$message` defaults to `null`. Call it *with* a message → write into session (set mode). Call it *without* → read and immediately `unset()` (get mode). The unset is the whole point: a flash is shown once, then gone — reload the page and the green "Entry updated." banner doesn't come back. `??` keeps get-mode safe when there's nothing set (no warning, just null). This is how a page that `redirect()`s — ending its own request — can still show a message in the *next* request: the message rode the session across the gap from part 1.

The CSRF trio (81–103) — quick pass now, full story in part 10

```
function csrf_token(): string          // make (once) or return the session's secret random string
{
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function csrf_field(): string          // hidden input to embed in every form
{
    return '<input type="hidden" name="csrf_token" value="' . e(csrf_token()) . '">';
}

function csrf_verify(): void           // on POST: compare form value vs session value
{
    $token = $_POST['csrf_token'] ?? '';
    if (!is_string($token) || !hash_equals($_SESSION['csrf_token'] ?? '', $token)) {
        http_response_code(419);
        exit('Page expired — please go back and try again.');
    }
}
```

`random_bytes(32)` is a cryptographically-secure random generator; `bin2hex` renders it as 64 hex characters. `hash_equals` compares the attacker's guess to the secret *without timing side-channel* (plain `===` on strings returns early at the first differing byte — theoretically observable). Status 419 is a non-standard "Page Expired" (popularised by Laravel) — any 4xx would do; dying immediately is the point. All three functions exist because every mutating form in the app carries the field and calls the verifier. Part 10 ties this to the actual attack.

The two date helpers (106–120)

```
function day_label(string $date): string
{
    return date('D d M Y', strtotime($date));
}
```

`'2026-09-14'` → `strtotime` parses to a timestamp (with Lagos timezone from config!) → `date('D d M Y')` formats → **"Mon 14 Sep 2026"**. Every date you see rendered in the UI passes through here.

```
function siwes_week_number(array $student, string $date): int
{
    $stmt = db()->prepare('SELECT MIN(entry_date) FROM log_entries WHERE student_id = ?');
    $stmt->execute([$student['id']]);
    $first = $stmt->fetchColumn();
    if (!$first) return 1;
    $weeks = (int)floor((strtotime($date) - strtotime((string)$first)) / 604800);
    return max(1, $weeks + 1);
}
```

"Which SIWES week is this?" — the student's *earliest* entry is week 1; every 604,800 seconds (= 7×24×3600, a week) advances it. `MIN()` with no rows returns `NULL` → `fetchColumn()` gives false → week 1. `max(1, …)` clamps against dates *before* the first entry (negative weeks). One subtlety worth seeing in logs.php: this runs **one extra SQL query per rendered row** — a classic "N+1". Fine for one student's dozens of rows; a scaling conversation to raise if asked.

What does the `static $pdo = null;` line accomplish?

- Reuses one connection across calls
- Shares a connection across all requests
- Opens a fresh connection on each call

Why does `WHERE email = ?` with execute([$email]) block SQL injection?

- Structure and data ship separately
- The placeholder strips quotes from input
- PDO escapes the string before sending it

What does `PDO::ATTR_EMULATE_PREPARES => false` choose?

- Real server-side prepared statements
- Faster string-built queries locally instead
- Hashing user input server-side before saves

In `current_student()`, what does `$stmt->fetch() ?: null` do?

- Converts no-row false into clean null
- Converts the student id into a nullable string
- Converts database errors into empty arrays

Why doesn't a flash message come back on the next page load?

- Reading it deletes it from session
- The browser blocks message caching
- PHP expires session keys per request

Why compare CSRF tokens with `hash_equals()`, not `===`?

- To hide timing clues from attackers
- To accept hashes instead of plain strings
- To compare case-insensitively by default

---

4 The partials — icons, layout-top, layout-bottom

Logged-in pages don't repeat their chrome. Instead each page is a three-act sandwich:

```
$page_title = 'Dashboard';
$active     = 'dashboard';
require __DIR__ . '/partials/layout-top.php';   // <!doctype> … sidebar … open <main>
```

```
?>
   …page-specific HTML here…
<?php
require __DIR__ . '/partials/layout-bottom.php'; // close <main>, footer, </html>
```

The variable-setting lines are an **implicit contract**: layout-top.php reads `$page_title` and `$active` because `require` runs the file *in the caller's variable scope* — the partial sees everything defined above it, and anything it defines leaks back. It's not magic, it's just lexical scope — but it's why the partials never work if you forget to set the variables first (they defend themselves: `$page_title = $page_title ?? APP_NAME;`).

icons.php — a function that returns SVG

```
function icon(string $name, string $class = 'icon'): string
{
    $paths = [
        'home' => '<path d="M3 10.5 12 3l9 7.5"/>…',
        // 20 more entries
    ];

    $d = $paths[$name] ?? $paths['check'];
    return '<svg class="' . e($class) . '" viewBox="0 0 24 24" …>' . $d . '</svg>';
}
```

A lookup array of SVG fragments keyed by name; `icon('trash')` returns the complete `<svg>` string — an SVG is just XML between two tags, stroke-based paths that inherit the text colour via `currentColor`. `string $class = 'icon'` is a **default parameter** — call with one argument and it fills in the rest. Unknown names fall back to the check icon (`??`) rather than crashing. The whole icon "library" is one function and an array — no image files anywhere.

layout-top.php — the shell

```
require_once __DIR__ . '/../helpers.php';

$page_title = $page_title ?? APP_NAME;
$active     = $active     ?? '';
$student    = current_student();
$flashError = flash('error');
$flashSuccess = flash('success');

$navItems = [
    'dashboard' => ['Dashboard',  icon('home')],
    'logs'      => ['My Log Book', icon('book')],
```

```
    'profile'   => ['Profile',     icon('user')],
];
```

The top of the file is all preparation, and it's instructive: it re-requires helpers (harmless `_once`), applies `??` defaults for the contract variables, then **consumes the flashes right now** (`flash('error')` — get mode — reads-and-deletes into local variables). That's why you never see a flash function called again lower down: they were read into `$flashError` / `$flashSuccess` before the HTML started. The nav is one array: key = page name, value = [label, svg]. Then:

```
<?php foreach ($navItems as $key => [$label, $svg]): ?>
    <a class="sidebar__link <?= $active === $key ? 'is-active' : '' ?>"
       href="<?= e($key) ?>.php" <?= $active === $key ? 'aria-current="page"' : '' ?>>
        <?= $svg ?><span><?= e($label) ?></span>
    </a>
<?php endforeach; ?>
```

Three constructs in three lines: `foreach … as $key => [$label, $svg]` — loop with key, and **destructure** the two-element array into named variables. `$active === $key ? 'is-active' : ''` — ternary deciding whether this link is highlighted (the active page passes its key in `$active`; remember the contract). `href="<?= e($key) ?>.php"` — the nav's keys are *the filenames themselves*: 'logs' → `logs.php`.

Lower in the sidebar: `<?= e(strtoupper(mb_substr($student['full_name'] ?? 'S', 0, 1))) ?>` — the avatar. `?? 'S'` survives a null student (deleted account mid-session) without erroring; `mb_substr(…, 0, 1)` = first character, multibyte-safe (`mb_` functions understand UTF-8 characters; plain `substr` counts bytes and would mangle accented names); `strtoupper` capitalises. Then the flash banners render, guarded: `<?php if ($flashSuccess): ?>` — alternative syntax: a colon-ended `if` inside HTML, closed by `endif;`. PHP supports either style; inside templates the colon form reads better.

layout-bottom.php

```
    </main>

    <footer class="footer">
        <small>&copy; <?= date('Y') ?> <?= e(APP_NAME) ?> — Built for SIWES log book management.</small>
    </footer>
</div>
<script src="assets/js/app.js"></script>
```

```
</body>
</html>
```

`©` is the © HTML entity; `date('Y')` the self-updating year; then the single script tag for the whole app (theme toggle + mobile sidebar + password eye — client-side sugar only; **every real behaviour of the app works without JavaScript**). Notice there is no `<?php` at its start — the file is HTML that ends in two PHP expressions. A PHP file doesn't even need to contain PHP.

How does layout-top.php know `$page_title` is 'Dashboard'?

- Require runs it in the caller's scope
- Pages pass it as a function argument
- Sessions store titles between each load

Why does layout-top call flash() at the top, storing results in locals?

- Reading early deletes it for later
- Headers must set before body output starts
- Session reads need a database connection

Why `mb_substr` instead of plain `substr` for the avatar letter?

- Byte-based cuts break accented letters
- Multibyte functions also run faster here
- Plain substr cannot take a zero position

5 database.sql — the schema and every constraint

The app has exactly two tables. Read `database.sql` alongside this; every line is a decision with a reason.

```
CREATE DATABASE IF NOT EXISTS siwes_tracker
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

`utf8mb4` = the *real* UTF-8 for MySQL (the legacy `utf8` charset maxes at 3 bytes per character — no emoji, some scripts). `utf8mb4_unicode_ci` = collation (the rule set for comparing strings); `_ci` is case-*insensitive*: `'a@x.com' = 'A@X.COM'` is TRUE in comparisons. Remember this — it makes the email uniqueness below case-proof too. (CREATE TABLE — MySQL Manual)

```
CREATE TABLE IF NOT EXISTS students (
  id            INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  matric_no     VARCHAR(40)  NOT NULL UNIQUE,
  full_name     VARCHAR(120) NOT NULL,
  email         VARCHAR(160) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  institution   VARCHAR(160) NULL,
  … (course, placement org, supervisor) …
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Line by line, the vocabulary of constraints:

- `INT UNSIGNED`: whole numbers, no sign bit — negative ids impossible, positives double-range.
- `AUTO_INCREMENT PRIMARY KEY`: MySQL assigns `1, 2, 3…` automatically. The PK is *also* physically how InnoDB stores the rows (part-2 knowledge: the clustered index) and the value PDO tells you about after an insert — `$pdo->lastInsertId()` in seed.php.
- `NOT NULL` vs `NULL`: institution etc. may be empty (registered later via profile); name/email can't. `NULL` in SQL means "absent", not "empty string" — a distinction you saw PHP-side when `fetch()` returned `NULL`s needing `?: 'Not set'` on screen.
- `UNIQUE` on matric and email: the database **refuses a second row** with the same value, full stop — even under a race where two registrations pass the PHP check (part 7). UNIQUE is an index; the refusal beats any PHP you write.

- `password_hash VARCHAR(255)`: a bcrypt hash (`$2y$10$…`) is 60 chars; 255 is room for future algorithms (argon2id). The DB must never hold a plain password — login.php can verify without ever seeing the secret because of how these hashes work (part 6).
- `TIMESTAMP DEFAULT CURRENT_TIMESTAMP` / `… ON UPDATE …`: MySQL itself stamps rows on insert, and re-stamps on every update. Free audit columns — PHP never sets them.
- `ENGINE=InnoDB`: the transactional, FK-supporting storage engine (the old MyISAM can't do constraints).

```
CREATE TABLE IF NOT EXISTS log_entries (
  id            INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  student_id    INT UNSIGNED NOT NULL,
  entry_date    DATE NOT NULL,
  hours_worked  DECIMAL(4,1) NOT NULL DEFAULT 8.0,
  work_done     TEXT NOT NULL,
  challenges    TEXT NULL,
  remarks       TEXT NULL,
  … timestamps …
  FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE,
  UNIQUE KEY uq_student_day (student_id, entry_date)
) ENGINE=InnoDB;
```

- `DECIMAL(4,1)`: exact fixed-point — 4 digits, 1 after the point (max 999.9, and the app caps at 24). Why not a float? Floats store approximations; `0.5 + 0.5 + …` in binary can't land on exact hours. Money-and-hours values get DECIMAL. (PHP receives them as strings — hence every display casting `(float)`.)
- `TEXT`: long variable strings — no practical width for a day's narrative. (`work_done` is the one NOT NULL field; the rest are optional.)
- `FOREIGN KEY … REFERENCES students(id)`: student_id must point at a real student row — the DB, not PHP, enforces that no orphan log can exist. `ON DELETE CASCADE`: delete a student and MySQL deletes their entries automatically — referential integrity as policy.
- `UNIQUE KEY uq_student_day (student_id, entry_date)`: the *pair* must be unique (a student may log 2026-09-14 once; two students may each). This is the app's "one entry per day" rule's **real** enforcer — log_form.php's friendly check is the courtesy layer on top. A composite index also speeds up the app's most common query (`WHERE student_id = …`) — its leftmost column.

The `logs.php` search uses `work_done LIKE '%term%'` — and here the two halves of this course meet: a *leading* wildcard can't use any index (there's no sorted structure to enter at), so that search scans. At one student's scale, irrelevant; at ten thousand students', a real conversation. You now know both layers — SQL and PHP — well enough to have that conversation in a viva.

What actually guarantees no student logs one date twice?

- The composite unique key blocks it
- The friendly PHP pre-insert check catches it
- The cascade delete option prevents duplicates

A student row is deleted in phpMyAdmin. What happens to their entries?

- MySQL removes them automatically together
- They remain as orphaned rows forever
- The delete is refused unless empty first

Why `utf8mb4` rather than plain `utf8`?

- Legacy utf8 cannot store some characters
- mb4 sorts text faster than legacy utf8
- Legacy utf8 breaks primary key values

Why `DECIMAL(4,1)` for hours instead of a float type?

- Exact digits avoid rounding drift
- It indexes much faster than floats
- Floats reject half-hour values entirely

---

6 login.php, register.php, logout.php — authentication

The shape of a page that handles its own form

`login.php, register.php and profile.php all share the pattern that defines plain-PHP web apps:` **`one file answers both halves of a conversation.`**

- **GET** (you open the page): the `if (REQUEST_METHOD === 'POST')` block is skipped, the HTML renders with empty/errors-free form. The page *displays*.
- **POST** (you hit Submit, the same file): the block at the top runs — validate, maybe write DB, maybe redirect. Only if validation *fails* does execution continue into the HTML, which now re-renders with errors shown and `$_POST` values echoed back.

That's why `$_SERVER['REQUEST_METHOD']` gates everything, and why form fields print `value="<?= e($_POST['email'] ?? '') ?>"` — on failed submit, your typing is restored (escaped, as always, because a POST value is user data). login.php even uses it *before* the POST block exists, as its display state.

login.php, top half (lines 1–31)

```
require_once __DIR__ . '/helpers.php';

if (is_logged_in()) { redirect('dashboard.php'); }

$error = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_verify();
    $email    = trim($_POST['email'] ?? '');
    $password = $_POST['password'] ?? '';

    if ($email === '' || $password === '') {
        $error = 'Please enter both email and password.';
    } else {
        $stmt = db()->prepare('SELECT * FROM students WHERE email = ?');
        $stmt->execute([$email]);
        $student = $stmt->fetch();

        if ($student && password_verify($password, $student['password_hash'])) {
            session_regenerate_id(true);
            $_SESSION['student_id'] = (int)$student['id'];
            $_SESSION['student_name'] = $student['full_name'];
            redirect('dashboard.php');
        }
        $error = 'Invalid email or password.';
    }
}
```

- **Reverse guard**: if already logged in, don't show the form — push to dashboard. (register.php copies it.)
- `trim()` eats stray spaces; `??` makes a missing key null-safely default; `$password` deliberately *not* trimmed (spaces may be part of it).
- The **email lookup uses the index** created by `UNIQUE email` — the login query is a fast `const`-type lookup, not a scan (nice schema↔code connection).

- `$student && password_verify(...)`: short-circuit — the expensive hash check only runs if the row exists. And here's the part-5 promise kept: password_verify recomputes the bcrypt hash from the typed password *using the salt embedded in the stored hash* (the `$2y$10$N9qo8uLO…` format carries algorithm, cost, salt, and digest in one string) and compares. The plain password was never in the database to read.
- **`session_regenerate_id(true)`**: at the exact moment the browser's identity upgrades from anonymous to "this student", hand it a brand-new session id (and destroy the old session file). Prevents *session fixation*: an attacker who planted a known session id before login can't ride it afterwards. (Manual)
- The two `$_SESSION` writes **are** the login. `student_name` is convenience for the UI; `student_id` is the credential everything else checks. `(int)` cast — the session can't be tricked into holding an array.
- **One error message for both failure modes**: "Invalid email or password" whether the email or the password was wrong. Telling the truth ("no such email") hands account enumerators a list of registered students. Generic-on-purpose.

register.php — validation, the uniqueness race, the insert

```
$errors = [];
$old    = fn(string $k) => e($_POST[$k] ?? '');
```

An **arrow function** stored in a variable, reused throughout the form for redisplaying submitted values — the `e()` built *into* the helper so a field can't be echoed unescaped even by mistake.

```
$matric   = strtoupper(trim($_POST['matric_no'] ?? ''));
…
if ($matric === '')                             $errors['matric_no'] = 'Matric number is required.';
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) $errors['email']     = 'Enter a valid email address.';
if (strlen($password) < 8)                      $errors['password']  = 'Password must be at least 8 chara
if ($password !== $confirm)                     $errors['password_confirm'] = 'Passwords do not match.';
```

Errors collected in an **associative array keyed by field name** (`$errors['email']`) — the form prints each under its field via `<?php if (isset($errors['email'])): ?>…<?php endif; ?>`. `strtoupper` normalises matric before anything else (CSC/2021/0123 == csc/2021/0123). `filter_var` with FILTER_VALIDATE_EMAIL checks shape only — it never confirms the inbox exists, which is why the app relies on the unique-email check instead. `strlen($password) < 8`: one rule, byte length.

```
if (!$errors) {
    $stmt = db()->prepare('SELECT email, matric_no FROM students WHERE email = ? OR matric_no = ?');
    $stmt->execute([$email, $matric]);
    while ($row = $stmt->fetch()) {
        if (strcasecmp($row['email'], $email) === 0)      $errors['email']     = 'This email is already r
```

```
        if (strcasecmp($row['matric_no'], $matric) === 0) $errors['matric_no'] = 'This matric number is a
    }
}
```

One query for both possible clashes (`OR`); `while ($row = $stmt->fetch())` is the standard PDO row-cursor loop (fetch returns false past the end, stopping the loop); `strcasecmp` compares case-insensitively in *PHP* so the right error lands under the right field, mirroring the DB's `_ci` collation. The friendly UX layer.

```
if (!$errors) {
    $stmt = db()->prepare('INSERT INTO students (matric_no, full_name, email, password_hash, institution,
        VALUES (?, ?, ?, ?, ?, ?)');
    $stmt->execute([$matric, $fullName, $email, password_hash($password, PASSWORD_DEFAULT), $institution,
    flash('success', 'Account created — you can now log in.');
    redirect('login.php');
}
```

`password_hash($password, PASSWORD_DEFAULT)`: generates a fresh random salt, runs bcrypt (the current `PASSWORD_DEFAULT`), returns the `$2y$10$…` string — **this is the only place the plain password is ever referenced, and it's never stored.** Then the PRG pattern: flash + redirect (part 7 names it). **The race worth knowing:** between the SELECT check and the INSERT, a second concurrent registration could slip past PHP — and then the `UNIQUE` key throws a PDOException, visible as an ugly 500. Belt (friendly check) and braces (the constraint): the DB is the one that's always right.

logout.php — the whole cycle in three lines

```
require_once __DIR__ . '/helpers.php';
$_SESSION = [];
session_destroy();
redirect('login.php');
```

`$_SESSION = []` empties the array; `session_destroy()` wipes the server-side storage. The cookie itself isn't deleted, it simply now points at nothing. Note what it assumes: `require_once` has already started the session (config.php), so there is something to destroy. The next request to dashboard.php: session exists, no `student_id` → `require_login()` → flash + redirect to login. The machine from part 1, end to end.

How can login check a password the database doesn't store?

- The hash embeds a salt for rechecking
- PHP keeps a decrypt key on the server
- Sessions cache every password once seen

Why "Invalid email or password" as one generic message?

- To deny attackers account-existence probes
- To simplify the PHP branch logic greatly
- To match messages users see on other sites

What attack does `session_regenerate_id(true)` after login stop?

- Session fixation with a planted older id
- Cross-site request forgeries via forms
- Cookie theft across http to https links

Two simultaneous registrations with one email. What is the final defence?

- The database UNIQUE constraint rejects
- The duplicate-check select query above it
- The CSRF token will differ between them

---

7 CRUD — log_form.php, logs.php, log_delete.php, profile.php

log_form.php — one file, two modes: Create and Update

```
$id    = isset($_GET['id']) ? (int)$_GET['id'] : 0;
$entry = null;
```

```
if ($id > 0) {
    $stmt = db()->prepare('SELECT * FROM log_entries WHERE id = ? AND student_id = ?');
    $stmt->execute([$id, $student['id']]);
    $entry = $stmt->fetch();
    if (!$entry) { flash('error', 'Entry not found.'); redirect('logs.php'); }
}
```

The mode switch is the URL: `log_form.php` = new entry; `log_form.php?id=5` = edit entry 5. The `(int)` cast is load-bearing: `?id=abc` becomes `0` (create mode), `?id=5%20OR%201=1` becomes `5` — a scalar can never smuggle SQL. Then the most important sentence in the file: **`WHERE id = ? AND student_id = ?`**. The lookup — like every query that touches a single entry in this app — is *scoped to the logged-in student*. Editing another user's entry #5 by typing its URL is impossible by design: no row matches, "Entry not found". The pattern has a name: Insecure Direct Object Reference (IDOR), and this schema-per-query is its textbook cure. (Hands-on below — you'll try to break it.)

```
$entryDate   = $entry['entry_date']   ?? date('Y-m-d');
$hoursWorked = $entry['hours_worked'] ?? 8.0;
$workDone    = $entry['work_done']    ?? '';
```

The defaults trick: fields start from the existing row in edit mode, or sane blanks in create mode. Note that `$_POST` handling (below) *overwrites* these, so a failed submit re-shows what you typed — one variable set feeds both the defaults and the redisplay. Clean.

```
if (!preg_match('/^\d{4}-\d{2}-\d{2}$/', $entryDate) || !strtotime($entryDate)) {
    $errors['entry_date'] = 'Pick a valid date.';
}
if ($hoursWorked < 0.5 || $hoursWorked > 24)  $errors['hours_worked'] = 'Hours must be between 0.5 and 24
if ($workDone === '')                         $errors['work_done']    = 'Describe what you did this day.'
```

A regex enforces the exact shape `YYYY-MM-DD` (that's what `<input type="date">` submits), *and* `strtotime` confirms it's a real calendar date — `2026-02-30` passes the pattern but fails the parse. Why the double check: shape is cheap and rejects garbage; `strtotime` is authoritative.

```
$stmt = db()->prepare(
    'SELECT id FROM log_entries WHERE student_id = ? AND entry_date = ? AND id <> ?'
);
$stmt->execute([$student['id'], $entryDate, $id]);
if ($stmt->fetch()) {
    $errors['entry_date'] = 'You already logged this day — edit that entry instead.';
}
```

The friendly duplicate-day check, and the elegance of `id <> ?` ("id is not this one"): in *create* mode `$id` is `0` so no row is excluded; in *edit* mode the row being edited is excluded from the clash — saving the entry unchanged passes. This query also needs no `fetchColumn` — `SELECT id`, truthy row existence is the whole answer.

```
if ($id > 0) {
    $stmt = db()->prepare('UPDATE log_entries SET entry_date = ?, hours_worked = ?, work_done = ?, challe
         WHERE id = ? AND student_id = ?');
    $stmt->execute([$entryDate, $hoursWorked, $workDone, $challenges, $remarks, $id, $student['id']]);
    flash('success', 'Entry updated.');
} else {
    $stmt = db()->prepare('INSERT INTO log_entries (student_id, entry_date, hours_worked, work_done, chal
         VALUES (?, ?, ?, ?, ?, ?)');
    $stmt->execute([$student['id'], $entryDate, $hoursWorked, $workDone, $challenges, $remarks]);
    flash('success', 'Day logged — nice progress!');
}
redirect('logs.php');
```

Both branches carry the scope guard (`WHERE … AND student_id = ?` — double protection: even if `$id` were someone else's row, the UPDATE touches zero rows) and both end in the same two moves: `flash()` then `redirect()`. That's the Post/Redirect/Get pattern: after any successful POST, never render — bounce to a GET page. Payoff: pressing Back or Refresh on logs.php can't resubmit the form; you get the list plus the "Day logged!" flash, every time. (The failure path deliberately skips PRG — it re-renders the form with `$errors` so your typing isn't lost.)

logs.php — Read, with a LIKE search

```
$q = trim($_GET['q'] ?? '');

if ($q !== '') {
    $stmt = db()->prepare(
        'SELECT * FROM log_entries
         WHERE student_id = ? AND (work_done LIKE ? OR challenges LIKE ? OR entry_date LIKE ?)
         ORDER BY entry_date DESC'
    );
    $like = '%' . $q . '%';
    $stmt->execute([$student['id'], $like, $like, $like . '%']);
} else {
    $stmt = db()->prepare('SELECT * FROM log_entries WHERE student_id = ? ORDER BY entry_date DESC');
    $stmt->execute([$student['id']]);
}
$entries = $stmt->fetchAll();
```

Search arrives as a URL query (`logs.php?q=git`) — that's what `<form method="get">` does: it builds the query string instead of a POST body, which makes searches **bookmarkable and shareable**. The `%` signs are built *in SQL-side data, not string-concatenated SQL* — `'%' . $q . '%'` is a value bound to a placeholder, so wildcards are added safely around user text. Two wildcard shapes: `%git%` = "contains" for the text fields; `2026-07%` (trailing only) for dates, so typing "2026-0" matches July entries. The `else` branch is just "show everything mine, newest first" — and note the *one* difference the search adds: `AND (LIKE …)`; `student_id` scoping is constant in both paths, as always. The table below then loops `$entries` with the same constructs as the dashboard; the empty state even branches on whether `$q !== ''` to say "No matches for …" versus "Nothing logged yet".

log_delete.php — never mutate on a GET

```
$id = (int)($_GET['id'] ?? 0);
$stmt = db()->prepare('SELECT * FROM log_entries WHERE id = ? AND student_id = ?');
$stmt->execute([$id, $student['id']]);
$entry = $stmt->fetch();
if (!$entry) { flash('error', 'Entry not found.'); redirect('logs.php'); }

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_verify();
    $stmt = db()->prepare('DELETE FROM log_entries WHERE id = ? AND student_id = ?');
    $stmt->execute([$id, $student['id']]);
    flash('success', 'Entry deleted.');
    redirect('logs.php');
}
// …GET falls through to the confirmation page HTML…
```

Two rules encoded here. **1. Confirmation before destruction:** GET shows "Delete this entry?" with a preview; only POST deletes. **2. Safe vs unsafe methods:** browsers pre-fetch links, crawlers follow them, and a sneaky `<img src="log_delete.php?id=5">` in a comment *would* fire — if deletion were a GET. Because deletion is POST-only *and* behind `csrf_verify()`, an attacker page can't forge that request either — the CSRF token would be missing, 419. This file is the single clearest demo of why CSRF matters, more than any other in the app.

In its HTML, one line earns study: `<?= nl2br(e(mb_strimwidth($entry['work_done'], 0, 300, '…'))) ?>` — read right-to-left: truncate to 300 display-widths with an ellipsis, **escape** for HTML, then convert newlines to `<br>`. The order escape-then-nl2br is mandatory: `nl2br` must run on already-escaped text, otherwise you'd feed raw user markup into the page — a neat, one-line XSS-avoidance lesson.

profile.php — two forms, one page, dispatched by a hidden field

```
$action = $_POST['action'] ?? '';
```

```
if ($action === 'details') {
    … validate full_name; UPDATE students SET full_name = ?, institution = ?, course_of_study = ?,
        placement_org = ?, industry_supervisor = ? WHERE id = ? …
    flash('success', 'Profile updated.'); redirect('profile.php');
}

if ($action === 'password') {
    if (!password_verify($current, $student['password_hash'])) { $errors['current_password'] = …; }
    if (strlen($new) < 8) { … } if ($new !== $confirm) { … }
    if (!$errors) { UPDATE students SET password_hash = ? WHERE id = ?; flash; redirect; }
}
```

Two `<form>`s on the page, each with a hidden `<input name="action" value="details|password">` — the server branches on it. It's the *same file* both forms submit to, so `$action` is how one POST endpoint serves two operations. Notice the details form leaves `email` and `matric_no` out of its UPDATE entirely — immutable by omission; and the matric is displayed in a `disabled` input (browser-greyed, and a disabled field is never submitted at all). The password branch starts by verifying the current password — change-your-password must prove-you're-you — and rehashes with a fresh `password_hash`.

Hands-on checkpoint (break it, ethically): register a *second* student account in another browser profile. Log in as student B, note their entry id from an edit URL (`log_form.php?id=9`), then paste that URL into student A's session. Watch what `WHERE id = ? AND student_id = ?` says on your behalf. Same test with `log_delete.php?id=9` .

What stops student A editing student B's entry via ?id= links?

- The scope clause added to every query
- The integer cast applied to id on arrival
- The CSRF token checked on every load

After a successful save, why redirect() instead of rendering directly?

- Refresh then cannot resubmit the post
- Rendering needs a fresh database read
- Post responses skip the flash system entirely

In the search, why is `'%' . $q . '%'` safe but concatenating into the SQL is not?

- It is a bound value, never query syntax
- Percents neutralise all injection payload
- Trim already removed any quote characters

Why must deletion only ever happen on POST, never GET?

- GETs get fired without any user consent
- GET bodies cannot carry the record id
- PHP silently blocks deletes from GETs

The delete preview escapes *then* nl2brs. Why can't that order flip?

- Escaping must precede new tag insertion
- nl2br cannot process any escaped content
- Browsers ignore line breaks otherwise entirely

---

8 dashboard.php — stats, array functions, the pure-CSS chart

The most compute-heavy page: four stat cards, a bar chart, a recent-entries table, built from three queries and some array gymnastics — and *zero* charting JavaScript.

```
$stmt = db()->prepare(
    'SELECT COUNT(*)                    AS entries,
            COALESCE(SUM(hours_worked), 0) AS hours,
            MIN(entry_date)             AS first_day,
            MAX(entry_date)             AS last_day
     FROM log_entries WHERE student_id = ?'
);
$stmt->execute([$student['id']]);
$stats = $stmt->fetch();
```

Four aggregates in one row — COUNT, SUM, MIN, MAX each collapse the student's rows to one value; `AS` names the result column so PHP can say `$stats['entries']`. Aggregate functions: `COALESCE(SUM(...), 0)` exists because `SUM` over *zero rows* is NULL — for a brand-new student the function must not print blank; `COALESCE(x, 0)` says "if NULL, use 0". One query, one row, four numbers: this is what "push computation into the database" looks like — you don't SELECT all entries and count them in PHP when MySQL already has an index that knows.

```
$entryCount  = (int)$stats['entries'];
$totalHours  = (float)$stats['hours'];
$avgHours    = $entryCount > 0 ? $totalHours / $entryCount : 0.0;
$weekCount   = 0;
if ($stats['first_day'] && $stats['last_day']) {
    $weekCount = (int)floor((strtotime((string)$stats['last_day']) - strtotime((string)$stats['first_day'
}
```

The casts are not decoration: PDO returns *everything as strings* (so does MySQL over its wire protocol) — `(int)"12"` is a real integer, safe for arithmetic and formatting. `$entryCount > 0 ? … : 0.0`: divide-by-zero guard, same instinct as COALESCE but PHP-side. Week span: same 604,800-seconds division as `siwes_week_number`, plus 1 because a student on their first day has spanned "week 1", not week 0; the `&&` guard skips the maths for empty logs.

```
$stmt = db()->prepare('SELECT entry_date, hours_worked FROM log_entries
     WHERE student_id = ? ORDER BY entry_date DESC LIMIT 10');
$stmt->execute([$student['id']]);
$chartRows = array_reverse($stmt->fetchAll());

$chartLabels = array_map(fn($r) => date('d M', strtotime($r['entry_date'])), $chartRows);
$chartData   = array_map(fn($r) => (float)$r['hours_worked'], $chartRows);
```

SQL says "give me the 10 newest" (`ORDER BY … DESC LIMIT 10` — `uq_student_day`'s index makes that fast); array_reverse puts them oldest→newest for the left-to-right chart. Two array_map calls with `fn() =>` arrow functions then split one array of rows into two parallel arrays: labels ("03 Sep") and numbers. The functional idiom: *transform every element, preserve position*.

```
<?php
$max = max($chartData) ?: 1;
foreach ($chartData as $i => $val):
    $pct = round($val / $max * 100);
?>
<div class="chart__col" title="… hours">
    <span class="chart__val"><?= (float)$val ?></span>
    <div class="chart__track"><div class="chart__bar" style="--h: <?= $pct ?>%"></div></div>
    <span class="chart__label"><?= e($chartLabels[$i]) ?></span>
```

```
    </div>
    <?php endforeach; ?>
```

The chart, explained fully in the HTML it prints. `max($chartData) ?: 1`: the tallest bar becomes the 100% reference (the elvis guards an all-zero — or empty — set from dividing by zero). Each bar's height is *relative to the max*, as a percentage, and the only place CSS appears in this course: `style="--h: 88%"` sets a custom property on the element; `assets/css/style.css` has `.chart__bar { height: var(--h); }` — PHP computes the data, CSS draws the bar. No canvas, no library, just math → percentage → a CSS variable per bar. (The `title` attribute is a native browser tooltip — accessibility for free; the `aria-label` a few lines up describes the whole chart to screen readers, built by `implode(', ', array_map(fn($l, $v) => …, $chartLabels, $chartData))`: array_map can walk *two arrays at once*, and implode joins with commas.)

```
<h2>Hello, <?= e(explode(' ', trim((string)$student['full_name']))[0]) ?></h2>
```

The greeting. `explode(' ', $name)` splits on spaces into an array; `[0]` is the first element — the first name. Note the `[0]` applied *directly to a function call* (legal since PHP 5.4): `explode(…)[0]`.

```
<td>
    <strong><?= e(day_label($row['entry_date'])) ?></strong>
    <small class="muted block">Week <?= siwes_week_number($student, $row['entry_date']) ?></small>
</td>
```

The recent-entries table loops the third query's `LIMIT 5` rows with `foreach … endforeach`. Here, part 3's warning pays out: `siwes_week_number()` fires its `MIN(entry_date)` query *inside the loop, per row* — five rows, five extra queries. Correct, readable, and the exact shape of an N+1 you'd fix by computing `$first` once above the loop. Knowing the fix before you need it is viva-level.

And the page's last trick: singular/plural UI text without any special feature — `Day<?= $entryCount === 1 ? '' : 's' ?> logged`. Ternaries, everywhere, inline.

What would a new student's total-hours card print without COALESCE?

- Nothing — sum of nothing is NULL
- Zero — sums always start at zero
- An error page from the failed query itself

Why COUNT/SUM in SQL instead of fetching rows and adding them in PHP?

- The database computes aggregates closer
- PHP cannot add decimal numbers up easily
- Prepared statements forbid returning every row

What job are the two `array_map(fn($r) => …)` lines doing?

- Projecting rows into parallel arrays
- Filtering out the rows with no values
- Sorting rows by date then hours descending

How does `style="--h: <?= $pct ?>%"` produce a bar height?

- CSS reads it via var() for height
- JavaScript animates the property later
- It is a valid CSS height directly

---

9 seed.php & router.php — the support files

seed.php — the app's only try/catch

Remember config.php's `ERRMODE_EXCEPTION`: a PDO error now *throws*. seed.php is the one file that catches it:

```
try {
    $pdo = db();
    $pdo->exec('DELETE FROM log_entries');
    $pdo->exec('DELETE FROM students');
    …
    $stmt = $pdo->prepare('INSERT INTO students (…) VALUES (?, ?, ?, ?, ?, ?, ?, ?)');
    $stmt->execute([…, password_hash('demo1234', PASSWORD_DEFAULT), …]);
    $studentId = (int)$pdo->lastInsertId();
```

```
    $weeks = [ [ ['2026-07-06', 8.0, "Set up the dev environment…", 'Getting used to…'], … ], … ];

    $stmt = $pdo->prepare('INSERT INTO log_entries (student_id, entry_date, hours_worked, work_done, chal
         VALUES (?, ?, ?, ?, ?)');
    foreach ($weeks as $week) {
        foreach ($week as $day) {
            [$date, $hours, $work, $challenges] = array_pad($day, 4, null);
            $stmt->execute([$studentId, $date, $hours, $work, $challenges]);
        }
    }
    echo '<h2>✅ Demo data created</h2>';
} catch (Throwable $ex) {
    http_response_code(500);
    echo 'Seeding failed: ' . htmlspecialchars($ex->getMessage());
}
```

- `exec()`: the simplest PDO method — run one static statement, no parameters, so no placeholders needed. Safe *here* because the SQL is literally fixed; elsewhere the app always prepares.
- `lastInsertId()`: MySQL reports the AUTO_INCREMENT id it assigned (part 5's connection) — the FK needs it on every entry.
- `$weeks` is an **array of arrays of arrays**: five weeks, each of five days, each day a 3- or 4-element list. The inner loop's `[$date, $hours, $work, $challenges] =` is *list destructuring* — unpack a positional array into named variables. `array_pad($day, 4, null)` pads short days to 4 elements so the destructuring never errors (days without challenges).
- **One prepare, 25 executes**: the statement compiles once; each execute sends fresh bound data. This is the efficiency the prepare step buys.
- `catch (Throwable $ex)`: catch any PHP error *or* exception (Throwable is the root of both trees — Manual); 500 status + `htmlspecialchars($ex->getMessage())` — even error text is escaped; the file itself instructs: delete it after seeding, because it's a public "re-create a demo password" button if left on a server.

router.php — only used by `php -S`, not XAMPP

```
$uri  = urldecode(parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH) ?? '/');
$file = __DIR__ . $uri;

if ($uri !== '/' && file_exists($file) && !is_dir($file)) {
    return false;                       // 1. serve static files ourselves
}
if ($uri === '/' || $uri === '') {
    require __DIR__ . (is_logged_in() ? '/dashboard.php' : '/login.php');
    return true;                        // 2. root routes by auth state
}
$candidate = __DIR__ . rtrim($uri, '/') . '.php';
```

```
if (file_exists($candidate)) {
    require $candidate;
    return true;                          // 3. /logs  → logs.php
}
http_response_code(404);
echo '<h1>404</h1>…';                      // 4. anything else
return true;
```

XAMPP ignores this file entirely — it exists because `php -S localhost:8000 router.php` gives you a no-Apache dev server, and when handed a router script, that server's contract is: "run my file; if it `return false`s, you (the server) handle the request normally." Hence the four numbered branches: static assets (css/js) return false so PHP's built-in server ships them untouched; `/` dispatches by login state; pretty paths gain a `.php` suffix and exist-check; and the default is a real 404 status. `parse_url(…, PHP_URL_PATH)` peels off the `?q=…` query string; `urldecode` restores `%20`→space (the folder name has one); `rtrim($uri, '/')` drops trailing slashes. It also *requires helpers.php* — the only consumer outside the app pages, because branch 2 needs `is_logged_in()`. Under Apache, you'll note, you've always typed `logs.php` explicitly — this file is precisely what Apache's simpler rules don't give you.

Why does seed.php run `array_pad($day, 4, null)` before destructuring?

- Short days need a fourth filled slot
- Execute rejects any array of less than four
- Nulls pad ids for the foreign key column

Why prepare once then execute 25 times in the seeding loop?

- Compiling the statement happens once
- PDO cannot prepare two same-shaped queries twice
- Reusing one statement also keeps sessions

In router.php, what does `return false;` achieve for `assets/css/style.css`?

- The built-in server ships the file itself
- The request gets a silent 404 status code
- PHP compresses and caches the asset now

10 The security map — five defences, and the attacks they stop

Every line of "why" so far assembles into this table. This is the page to revisit the night before a viva.

Attack	What an attacker tries	Where the app blocks it
SQL injection	Type `' OR 1=1 --` in the login email to twist the query into returning the wrong row (or worse, `; DROP TABLE`).	Every query: `prepare()` + `execute([values])` — the `?` placeholder means data never enters the SQL grammar. `EMULATE_PREPARES => false` makes it MySQL's real engine enforcing this, not PHP string tricks. (part 3)
XSS (cross-site scripting)	Store `<script>fetch(evil)</script>` in a log's work_done, waiting for a viewer's browser to run it.	`e()` = htmlspecialchars on *every output* — the text renders as visible gibberish, never as markup. `ENT_QUOTES` closes the attribute-breakout hole. `trim` / `filter_var` narrow inputs but aren't the defence. (part 3)
CSRF	A hostile page submits a hidden delete/add form to your app using the visitor's unguessable-but-automatic session cookie.	`csrf_field()` embeds a session-bound random token in every form; `csrf_verify()` rejects mismatched POSTs with 419 before any work. An external page cannot read the token, so cannot forge the form. Backup: `samesite=Lax` cookie. (parts 2, 3)
IDOR (insecure direct object reference)	Change `?id=5` to `?id=9` to open or delete *someone else's* entry.	Single-entry queries always carry `AND student_id = ?` (and `WHERE id`-scoped on writes) — foreign rows don't match, "Entry not found". Break it yourself in part 7's checkpoint. (part 7)
Session theft / fixation	Sniff the session cookie, or plant a known session id before login to ride it after.	`httponly` cookie (script can't read it), and `session_regenerate_id(true)` at login rotates the id the instant it becomes valuable. Logout wipes the store. (parts 2, 6)

Plus three "policy" defences worth naming: passwords stored only as salted bcrypt (parts 5–6); one generic login error to deny enumeration; and deletion POST-only so GET-followers (crawlers, pre-fetchers, malicious `<img>` tags) can't mutate state.

CSRF exploits one fact about cookies. Which?

- Browsers attach session cookies automatically
- Cookies survive past the session end sometimes
- Hidden fields can store values without forms

Why does `' OR 1=1 --` typed into the login email do nothing?

- It arrives as data after query compilation
- Trim removes the dashes and the quotes both
- MySQL simply never parses double dashes at all

A stored log contains `<script>…</script>`. What does the viewer see?

- The literal tag text, displayed as characters
- Nothing at all — htmlspecialchars deletes it
- The script, but sandboxed safely by browsers

---

11 Viva drill — 14 questions, with model answers

`Cover each answer before reading it; answer out loud first. These are the questions an examiner who opens this folder will ask.`

▼ 1. Walk me through what happens when I load dashboard.php.

Apache hands the file to PHP. Line 1–2: `require_once helpers.php` → runs config.php (constants, `session_start()` loads my session; `$_SESSION['student_id']` proves I'm logged in) and icons.php. `require_login()` checks the session; passes. `current_student()` (cached) runs a PK lookup. Three more queries: the aggregate stats row, the last 10 dates, the 5 recent entries. PHP then interleaves computed values and `e()`-escaped data with HTML as it prints; the partials supply the shell. Output buffer becomes the response. The file, and all its memory, cease to exist.

▼ 2. Prepared statements — what are they and why every query here uses one?

SQL structure and user values travel separately: `prepare()` sends the grammar with `?`s (MySQL parses it immediately), `execute()` sends values typed as data. Values can never re-enter the parse, so injection is structurally impossible — which beats any escape routine as a strategy. `db()` sets `EMULATE_PREPARES=false` so it's MySQL doing this, not PHP faking it with quotes.

▼ 3. Where is the CSRF token stored, and what proves it?

Session (*server* side, in a file keyed to the cookie) and inside each form as a hidden field (*browser* side). On POST, `csrf_verify()` compares form value vs session value with `hash_equals()` (constant-time). Only this browser knows both halves — an external site's form can carry the user's automatic cookie but can never read the session's token, so its submission dies with 419.

▼ 4. Password security: what's in the database, and what happens at login?

60-character bcrypt strings: algorithm tag + cost (2^10 work) + random 16-byte salt + digest, all in one value, one row — `register.php`'s `password_hash()`. At login `password_verify()` re-runs bcrypt on the typed password with the stored salt and compares digests — the plain password is never stored or recovered. The salt makes identical passwords hash differently; the cost makes brute force ~0.1s/guess instead of ~0.000001s.

▼ 5. One entry per day — where is it enforced and why twice?

Database: `UNIQUE KEY uq_student_day (student_id, entry_date)` — absolute, race-proof. PHP: the `SELECT … id <> ?` check in log_form.php — friendly: turns a constraint error into a per-field message naming the date. The PHP check can't be the security boundary (TOCTOU race between its SELECT and INSERT); the constraint can. Belt and braces.

▼ 6. What's `$_SESSION` technically?

An ordinary array in the running script, populated at `session_start()` from a server file (`sess_<id>`) whose name came from the visitor's `PHPSESSID` cookie, and re-written at script end. It's a disk-backed mailbox between two separate executions of two different files — which is how logout can affect the *next* request's `is_logged_in()`, and how flash messages survive redirects.

▼ 7. How does one file serve both "new entry" and "edit entry"?

Presence of `?id=`: `$id = (int)$_GET['id'] ?? 0;` ; `$id > 0` triggers a scoped SELECT whose row supplies form defaults (with `??` fallbacks for create mode); the title, submit-label, and POST branch (UPDATE vs INSERT) all check `$id`. A single CRUD form, two modes.

▼ 8. What is the Post/Redirect/Get pattern and where does the app use it?

Every successful POST (login, register, save, delete) ends with `redirect()` to a GET page instead of rendering — so refresh resubmits nothing, and the confirmation is a session-riding `flash()` consumed by layout-top.php. Failed validations deliberately *don't* redirect — they re-render the form with `$errors` and echoed values.

▼ 9. Why can `<?=` be everywhere but `header()` only at file tops?

`<?=` appends to the response *body*; `header()` appends to the response *head*, and the head must already be sent before the first body byte. PHP buffers headers until output starts — `redirect()` calls therefore must precede any HTML in a file (the reason the app's PHP logic sits above its markup).

▼ 10. What's an N+1 query and where does it lurk here?

One query for the rows, then one more per row. `siwes_week_number()` runs `SELECT MIN(entry_date) …` *per rendered row* on dashboard and logs — the recent list costs 1 + 5. The fix: compute MIN once before the loop (or cache). Harmless at this scale, the first fix at scale.

▼ 11. Why are there only two tables, and how does the app draw "a student's entries"?

Relational: `log_entries.student_id` is the FK; "their" = `WHERE student_id = ?`, bound from `$_SESSION`. Normalisation: names once, not per-entry. The composite UNIQUE both enforces the daily rule and indexes the dominant query path. The FK guarantees integrity from the other direction.

▼ 12. Why does `fetch()` sometimes mean false, sometimes a row with nulls?

Zero rows → `false` (that's why `if (!$student)` works after login). Rows exist but nullable columns empty → array with `null` values. Hence the app's two idioms: `?: null` / `if (!$entry)` for *row existence*, and `?? 'default'` / `?: 'Not set'` for *empty fields*. Two different absences.

▼ 13. Where could this app improve if it went to production?

A good viva answer is layered: real DB user with password, no `root`; require login-over-HTTPS (secure cookie flag); rate-limit the login form (bcrypt cost per attempt is partial help); replace the register.php "email exists" message with a neutral one (it currently confirms accounts — a deliberate trade-off for this audience); fix seed.php deletion; the N+1; logging of PDO exceptions instead of raw 500s; a CSRF edge-case: tokens never rotate (acceptable, but worth naming). Showing you know the app's compromises *and* their reasons is the point.

▼ 14. Why PHP and not a framework?

Deliberate for this project: every mechanism visible. A framework would hide exactly the things examined here — sessions in middleware, CSRF in a template tag, PDO in an ORM. No Composer means `require_once` is the dependency system, and the 121-line helpers.php is a 20,000-word explanation of what Laravel does with `Artisan make:auth`.

12 Final exam — twelve mixed questions, from memory only

Interleaved across the whole course — the mix *is* the difficulty. Score first tries honestly; then paste any misses to me and we drill them.

PDO's `fetch()` with no matching row gives you what?

- False, not null and not an empty array
- Null, silently like the missing columns do
- An empty array, zero keys inside

Someone hits `log_form.php?id=abc`. What happens?

- Casts to zero, so it opens create mode instead
- Throws an error, the strict cast rejects all letters
- Passes through to SQL, enabling injection there

In register.php's form, what is `$old('email')`?

- An arrow function call, echoing safely
- A session lookup of what the email was before
- A database read of the old stored value now

A query fails. Why does the rest of the page not just quietly continue?

- ERRMODE_EXCEPTION throws it up loudly
- APACHE kills any PHP script on failure too
- PDO retries each failed statement automatically

Hit `http://localhost:8000/whatever` under php -S. Who says 404?

- router.php, after its .php candidate fails to exist
- helpers.php, through its require_login gate function
- Apache, from its own configured error document

Why must `$active = 'logs';` appear above the partial's require?

- Required files see the requiring scope's variables
- The partial exports its variables to each page
- Sessions carry those settings between each load

Where does the salt used by password_verify() come from?

- Inside the stored hash string itself
- From the sessions data per matching user
- A separate salt column of the students table

Which schema line makes 'my newest 10' queries fast?

- The unique day-key composite index ordering
- The auto-increment integer primary key itself
- The foreign key with its cascade setting

The error message you saw at login after require_login() — how did it cross the redirect?

- It rode in the session, consumed once there
- It travelled as a query string parameter
- It stayed alive in a static helper variable

Why `__DIR__ . '/helpers.php'` with a leading slash on the file part?

- __DIR__ never ends with a separator itself
- Leading slashes work for root-level files too
- require refuses backslashes inside all paths

In the UPDATE, why does `WHERE id = ? AND student_id = ?` repeat the guard?

- The write re-scopes, not just the read before
- The first guard ran against a different database
- UPDATE statements reject a single condition entirely

Which of these is TRUE about partials/layout-top.php?

- It consumes both flashes into local variables
- It closes the page by printing the footer tags
- It defines $navItems only when logged out today

> **How did you do?** 10+ first-try: you're viva-ready — ask me for a mock defence and I'll interrupt your answers like a real examiner. 6–9: name the misses, we build a one-page drill for them. Under 6: normal — the material is dense; we start a spaced-review plan from tomorrow's session. Either way, paste your score.

---

13 Primary sources & next steps

The high-trust readings behind this document, in the order they'd help you:

- PHP Manual — What is PHP? (https://www.php.net/manual/en/intro-whatis.php) — the 5-minute version of part 1, from the horse's mouth.
- PHP Manual — PDO (https://www.php.net/manual/en/book.pdo.php) — read the user notes on prepare/execute; part 3 expanded.
- PHP Manual — Sessions (https://www.php.net/manual/en/features.sessions.php) — part 2 and the $_SESSION mechanics, deeper.

- OWASP — CSRF Prevention Cheat Sheet (https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html) — part 10's third row, canonical.
- PHP: The Right Way (https://phptherightway.com/) — the wider context for every "why would anyone write it this way" you've had.

After this document: the references folder keeps the file map (../reference/file-map.html) and the glossary (../reference/php-glossary.html) — your look-up tables when a line goes rusty. For real retention, return in 2–3 days and take the part-12 exam cold (spacing turns fluency into storage), and use r/PHPhelp (https://www.reddit.com/r/PHPhelp/) or Stack Overflow when a question outgrows me.

A Answer key

One line per quiz question, in order. Quiz yourself from the parts before peeking here.

1. **Q1. Who executes the PHP in dashboard.php?**

   Answer: The PHP interpreter on the server

   Apache hands .php files to the PHP interpreter on the server; the browser receives only the printed HTML, never the source.

2. **Q2. Why can redirect() only run before any HTML prints?**

   Answer: Headers ship before the body does

   header() writes the redirect into the response's head, which ships before the body — after HTML output starts, PHP cannot add it.

3. **Q3. After a request finishes, where does a variable's value survive?**

   Answer: Nowhere — only session and database

   The script dies with the response. $_SESSION is a server file keyed to the visitor's cookie — the only PHP-visible state that survives between requests.

4. **Q4. Why are the DB settings constants rather than variables?**

   Answer: Constants cannot get overwritten later

   Constants created by define() cannot be reassigned; variables can. Settings like DB_HOST should be immutable for the request.

5. **Q5. Why must session_set_cookie_params() precede session_start()?**

   Answer: The cookie is built at start, once

   Cookie flags are baked into the session cookie when session_start() creates it. After that, the cookie is already in flight — order matters, exactly as the code shows.

6. **Q6. What do 'httponly' and 'samesite' defend against?**

Answer: Cookie theft and cross-site form posts

httponly blocks script access to the cookie; samesite=Lax stops it riding on cross-site POSTs. Both protect the session id from theft/abuse — a viva favourite.

7. **Q7. What does the `static $pdo = null;` line accomplish?**

   Answer: Reuses one connection across calls

   static $pdo keeps the object between calls: first call connects, later calls reuse. One connection per request, and only if needed (lazy).

8. **Q8. Why does `WHERE email = ?` with execute([$email]) block SQL injection?**

   Answer: Structure and data ship separately

   The SQL with its ? placeholders is prepared and sent first; values travel separately and are never parsed as SQL syntax. That separation is the injection defence.

9. **Q9. What does `PDO::ATTR_EMULATE_PREPARES => false` choose?**

   Answer: Real server-side prepared statements

   EMULATE_PREPARES=false turns off PHP's client-side fake prepares; statements go through MySQL's real prepare protocol — the genuine defence.

0. **Q10. In `current_student()`, what does `$stmt->fetch() ?: null` do?**

   Answer: Converts no-row false into clean null

   fetch() gives one row array or false; the elvis ?: turns false into null, matching the declared ?array return type.

1. **Q11. Why doesn't a flash message come back on the next page load?**

   Answer: Reading it deletes it from session

   Get-mode reads the key then unsets it — second call returns null. That's why the banner never reappears on reload.

2. **Q12. Why compare CSRF tokens with `hash_equals()`, not `===`?**

   Answer: To hide timing clues from attackers

   hash_equals compares in constant time — a timing attack on === could leak how many leading bytes of the secret match.

3. **Q13. How does layout-top.php know `$page_title` is 'Dashboard'?**

   Answer: Require runs it in the caller's scope

   require executes the file within the caller's symbol table — variables defined before the require line are visible inside, and vice versa.

4. **Q14. Why does layout-top call flash() at the top, storing results in locals?**

   Answer: Reading early deletes it for later

   flash('error') is get-mode: read + unset. The locals persist through the rest of this one request; the next request sees nothing.

5. **Q15. Why `mb_substr` instead of plain `substr` for the avatar letter?**

   Answer: Byte-based cuts break accented letters

   mb_substr counts characters in UTF-8; substr counts bytes. An initial 'Ọ' would be sliced into garbage by the byte version.

6. **Q16. What actually guarantees no student logs one date twice?**

   Answer: The composite unique key blocks it

   uq_student_day is a UNIQUE KEY: any INSERT/UPDATE making the pair exist twice fails, no matter which code path tried it.

7. **Q17. A student row is deleted in phpMyAdmin. What happens to their entries?**

   Answer: MySQL removes them automatically together

   CASCADE: MySQL deletes dependent log_entries rows whenever the students row goes — no orphans, no PHP cleanup needed.

8. **Q18. Why `utf8mb4` rather than plain `utf8`?**

   Answer: Legacy utf8 cannot store some characters

   utf8mb4 is full-width UTF-8 (4 bytes). Legacy 'utf8' is 3 bytes — it physically cannot store some characters people type.

9. **Q19. Why `DECIMAL(4,1)` for hours instead of a float type?**

   Answer: Exact digits avoid rounding drift

   DECIMAL stores digits exactly; FLOAT is binary-approximate (0.1+0.2 ≠ 0.3). Hour totals must add cleanly.

0. **Q20. How can login check a password the database doesn't store?**

   Answer: The hash embeds a salt for rechecking

   password_verify extracts algorithm+cost+salt from the stored string, re-hashes the candidate, and compares digests. The stored value was never reversible.

1. **Q21. Why "Invalid email or password" as one generic message?**

   Answer: To deny attackers account-existence probes

   Same message for wrong-email and wrong-password: the form can't be used to discover which addresses are registered.

2. **Q22. What attack does `session_regenerate_id(true)` after login stop?**

   Answer: Session fixation with a planted older id

   The pre-login session id may be known/planted by an attacker; rotating it at the privilege jump makes that stolen id dead.

3. **Q23. Two simultaneous registrations with one email. What is the final defence?**

   Answer: The database UNIQUE constraint rejects

If PHP's SELECT check loses a race, the UNIQUE constraint itself rejects the INSERT — DB enforcement beats any application check.

4. **Q24. What stops student A editing student B's entry via ?id= links?**

   Answer: The scope clause added to every query

   Every single-entry query carries AND student_id = ? — a foreign row simply never matches. Access control as a clause, on every path.

5. **Q25. After a successful save, why redirect() instead of rendering directly?**

   Answer: Refresh then cannot resubmit the post

   PRG: a successful POST answers with a redirect to a GET page. Refresh resubmits a harmless GET — no duplicate INSERT, and the flash proves it happened.

6. **Q26. In the search, why is `'%' . $q . '%'` safe but concatenating into the SQL is not?**

   Answer: It is a bound value, never query syntax

   Wildcards ride in as bound data, never as concatenated SQL text — placeholder discipline again, and search stays injection-proof.

7. **Q27. Why must deletion only ever happen on POST, never GET?**

   Answer: GETs get fired without any user consent

   Preloaders and hostile img tags fire GETs without the user; destructive actions must be POST (plus CSRF token), exactly as here.

8. **Q28. The delete preview escapes *then* nl2brs. Why can't that order flip?**

   Answer: Escaping must precede new tag insertion

   nl2br on escaped text adds only its own generated tags. Reverse the order and user-supplied markup gets injected.

9. **Q29. What would a new student's total-hours card print without COALESCE?**

   Answer: Nothing — sum of nothing is NULL

   SUM over zero rows yields NULL in SQL; COALESCE maps that one NULL into a printable 0 for a brand-new student.

0. **Q30. Why COUNT/SUM in SQL instead of fetching rows and adding them in PHP?**

   Answer: The database computes aggregates closer

   The DB collapses millions of potential PHP iterations into one row of numbers, using the index. Push work down where the data lives.

1. **Q31. What job are the two `array_map(fn($r) => …)` lines doing?**

   Answer: Projecting rows into parallel arrays

   array_map transforms each element positionally — rows become labels and rows become numbers, keeping order aligned.

2. **Q32. How does `style="--h: <?= $pct ?>%"` produce a bar height?**

Answer: CSS reads it via var() for height

--h is a custom property read by .chart__bar's height: var(--h). PHP supplies numbers; CSS renders geometry.

3. **Q33. Why does seed.php run `array_pad($day, 4, null)` before destructuring?**

   Answer: Short days need a fourth filled slot

   array_pad stretches short 3-element days to 4 with null, so the 4-variable destructure never hits a missing index.

4. **Q34. Why prepare once then execute 25 times in the seeding loop?**

   Answer: Compiling the statement happens once

   The prepare step parses/compiles the SQL server-side once; each execute only ships new bound values.

5. **Q35. In router.php, what does `return false;` achieve for `assets/css/style.css`?**

   Answer: The built-in server ships the file itself

   Returning false hands the request back to php -S's own static-file handling — the router only decides which PHP page runs, not the whole server.

6. **Q36. CSRF exploits one fact about cookies. Which?**

   Answer: Browsers attach session cookies automatically

   The browser sends the session cookie automatically on any same-site form post — CSRF's whole trick. The token can't be read cross-origin, so the forged form lacks it.

7. **Q37. Why does `' OR 1=1 --` typed into the login email do nothing?**

   Answer: It arrives as data after query compilation

   Placeholder values are bound as data after the SQL compiles; a quote inside them is just a character. No input can change the query's structure.

8. **Q38. A stored log contains `<script>…</script>`. What does the viewer see?**

   Answer: The literal tag text, displayed as characters

   e() converts < > & " to entities — browser sees text, not tags. The DB row stays original; only rendering is neutralised.

9. **Q39. PDO's `fetch()` with no matching row gives you what?**

   Answer: False, not null and not an empty array

   fetch() returns false for zero rows — both lookup sites test that exact signal before trusting a row.

0. **Q40. Someone hits `log_form.php?id=abc`. What happens?**

   Answer: Casts to zero, so it opens create mode instead

   The (int) cast turns 'abc' into 0 → create mode; a scalar string could never reach SQL anyway, but the cast makes id provably an integer.

1. **Q41. In register.php's form, what is `$old('email')`?**

   Answer: An arrow function call, echoing safely

   $old('matric_no') is an arrow function (register.php:9) — fn(string $k) => e($_POST[$k] ?? '').

2. **Q42. A query fails. Why does the rest of the page not just quietly continue?**

   Answer: ERRMODE_EXCEPTION throws it up loudly

   config.php sets ERRMODE_EXCEPTION: PDO throws instead of returning false — seed.php's try/catch depends on that one option.

3. **Q43. Hit `http://localhost:8000/whatever` under php -S. Who says 404?**

   Answer: router.php, after its .php candidate fails to exist

   The 404 comes from router.php's last branch: http_response_code(404) + a tiny echo — and only under php -S. Under Apache, this file never runs and Apache 404s unknown URLs itself.

4. **Q44. Why must `$active = 'logs';` appear above the partial's require?**

   Answer: Required files see the requiring scope's variables

   require runs in caller scope (part 4): variables set in the page above are visible inside layout-top.php — the contract of $page_title/$active.

5. **Q45. Where does the salt used by password_verify() come from?**

   Answer: Inside the stored hash string itself

   The 60-char bcrypt string: $2y$10$ + 22-char salt + 31-char digest. The salt lives inside the stored hash — that's how verify re-derives without extra columns.

6. **Q46. Which schema line makes 'my newest 10' queries fast?**

   Answer: The unique day-key composite index ordering

   uq_student_day(student_id, entry_date): its leftmost column is student_id, so 'WHERE student_id = ? ORDER BY entry_date' walks the index — fast list, fast DESC.

7. **Q47. The error message you saw at login after require_login() — how did it cross the redirect?**

   Answer: It rode in the session, consumed once there

   flash('error') in layout-top is get mode: read + unset. It was written by require_login() during a prior request — carried in $_SESSION across the redirect.

8. **Q48. Why `__DIR__ . '/helpers.php'` with a leading slash on the file part?**

   Answer: __DIR__ never ends with a separator itself

   __DIR__ never ends with a path separator, so the code supplies the '/' — gluing them directly would produce '…\siwes trackerhelpers.php'.

9. **Q49. In the UPDATE, why does `WHERE id = ? AND student_id = ?` repeat the guard?**

Answer: The write re-scopes, not just the read before

Belt-and-braces: the earlier SELECT gave the friendly error, but the UPDATE itself re-checks. A write scoped to the session's student_id can touch at most one's own rows, whatever happened before.

0. **Q50. Which of these is TRUE about partials/layout-top.php?**

Answer: It consumes both flashes into local variables

e(APP_NAME) — APP_NAME is the developer's own constant; escaping constants is the uniform 'every output goes through e()' habit. The < ?= is an echo tag; require_once is at line 13; the file does not end with ?>; and it reads $page_title from scope.

Anything in here that felt hand-wavy? Ask me — "what actually is bcrypt's cost factor?", "show me the sess_ file on my machine", "why `<>` instead of `!=`?". Every good question reorganises what you know.

References: PHP glossary · file map

This is the course's single lesson — back to top